

Plantilla Móvil Base (Mobile Starter)

Expo SDK 54 + React Native 0.81 + TypeScript + Clean Architecture & SOLID

1. Propósito y Beneficios de la Plantilla

Esta plantilla elimina por completo el tiempo repetitivo de inicialización de nuevos proyectos móviles. En lugar de reinstalar librerías de navegación, configurar temas oscuro/claro, tipar rutas, ajustar SafeArea y definir estructuras desde cero, tendrás una base lista para producción en **menos de 10 segundos**.

Beneficio	Antes (Configuración Manual)	Con la Plantilla Base
Tiempo de inicio	1 a 2 horas instalando y configurando.	10 segundos con 1 solo comando.
Arquitectura	Carpetas dispersas, acoplamiento alto.	Clean Architecture y SOLID estricto.
Diseño & Temas	Colores quemados, sin modo oscuro.	Sistema de temas (Light/Dark) persistido.
Almacenamiento	AsyncStorage llamado directo en pantallas.	IStorageService desacoplado (DIP/LSP).
TypeScript & Rutas	Strings mágicos y any frecuentes.	Navegación y props 100% tipados.

2. Estructura de Capas y Principios SOLID

El código está organizado siguiendo la separación estricta de responsabilidades:

Carpeta / Archivo	Capa	Responsabilidad & Principio
src/core/interfaces/	Dominio	DIP / LSP: Contratos abstractos como <code>IStorageService</code> y <code>IApiClient</code> .
src/core/types/	Dominio	Tipado estricto de rutas de navegación, temas y modelos comunes.
src/services/storage/	Infraestructura	Implementación concreta (<code>AsyncStorageAdapter</code>) con prefijo dinámico.
src/theme/	Diseño	Paletas Light/Dark, escalas tipográficas, espaciados y hook <code>useTheme()</code> .
src/components/	Presentación UI	Componentes atómicos reutilizables: <code>ScreenContainer</code> , <code>AppButton</code> , <code>AppTextInput</code> , <code>AppCard</code> , <code>AppHeader</code> .
src/screens/	Presentación	SRP: Vistas puras (<code>HomeScreen.tsx</code>) separadas de su lógica (<code>useHomeScreen.ts</code>).
src/navigation/	Navegación	Rutas declaradas como constantes (<code>ROUTES</code>) y <code>RootNavigator.tsx</code> .
src/utils/	Utilitarios	Funciones puras de formateo (<code>Formatters</code>) y validación (<code>Validators</code>).
src/config/	Configuración	Constantes de entorno (<code>env.ts</code>) y nombres centralizados.

3. Paso a Paso: Cómo Crear una Nueva Aplicación

Método 1: Creación Local Automática en 1 Comando (Recomendado)

La plantilla incluye el script automatizado `create-app.js`. Este script clona la plantilla, configura el nombre de la app, el slug, el package ID de Android, el Bundle ID de iOS y el prefijo de almacenamiento sin errores manuales.

Paso 1.1: Abre tu terminal PowerShell o CMD y navega a la carpeta de la plantilla:

```
cd c:\VibeCoding\AppFacturas\mobile-starter-template
```

Paso 1.2: Ejecuta el script generador con el nombre de tu nueva app y su Bundle ID:

```
node create-app.js "Mi App Delivery" com.miempresa.delivery
```

Paso 1.3: Entra a tu nuevo proyecto e instala dependencias:

```
cd ..\mi-app-delivery  
npm install
```

Paso 1.4: Inicia el entorno de desarrollo:

```
npx expo start
```

■ AUTOMATIZACIÓN COMPLETA

El script se encarga de cambiar automáticamente el `app.json`, `package.json` y `src/config/env.ts`. Tu nuevo proyecto queda completamente independiente y listo para codificar.

Método 2: Usar como Plantilla en GitHub

Si prefieres alojar la plantilla en tu cuenta de GitHub para que tú o tu equipo puedan clonarla desde cualquier máquina:

1. **Sube la carpeta** `mobile-starter-template` **a un nuevo repositorio GitHub** (ej: `tu-usuario/expo-starter-template`).
2. **En GitHub, entra a Settings y activa la casilla 'Template repository'.**
3. **Cada vez que quieras crear una nueva app, solo ejecutas desde cualquier terminal:**

```
npx create-expo-app MiNuevaApp --template https://github.com/tu-usuario/expo-starter-template
```

4. Guía de Desarrollo: Cómo Construir tu App

A continuación se detallan los flujos de trabajo recomendados para agregar pantallas, lógica, temas y almacenamiento respetando la arquitectura.

A. Cómo Personalizar Colores y Temas

Edita `src/theme/colors.ts` para definir la paleta de tu marca. El sistema incluye automáticamente modo claro y oscuro:

```
// src/theme/colors.ts
export const lightColors: ColorPalette = {
  primary: '#4F46E5',      // Color principal (Botones, acentos)
  secondary: '#0EA5E9',    // Color secundario
  background: '#F8FAFC',   // Fondo de pantallas
  surface: '#FFFFFF',      // Fondo de tarjetas y modales
  text: '#0F172A',        // Texto principal
  ...
};
```

Para consumir los colores en cualquier componente o pantalla, utiliza el hook:

```
const { colors, spacing, isDark, toggleTheme } = useTheme();
```

B. Cómo Crear una Nueva Pantalla (Aplicando SOLID)

Para mantener el principio de **Responsabilidad Única (SRP)**, crea siempre 2 archivos por pantalla en una carpeta dentro de `src/screens/`:

1. El Hook ViewModel (`usePerfilScreen.ts`): Contiene el estado, llamadas a APIs y lógica:

```
// src/screens/PerfilScreen/usePerfilScreen.ts
import { useState, useEffect } from 'react';
import { storageService } from '@services';

export function usePerfilScreen() {
  const [usuario, setUsuario] = useState('');
  const [cargando, setCargando] = useState(false);

  const guardar = async (nombre: string) => {
    await storageService.set('usuario_nombre', nombre);
    setUsuario(nombre);
  };

  return { usuario, cargando, guardar };
}
```

2. La Vista de Presentación (`PerfilScreen.tsx`): Solo renderiza UI limpia:

```
// src/screens/PerfilScreen/PerfilScreen.tsx
import React from 'react';
import { ScreenContainer, AppHeader, AppButton, AppTextInput, AppCard } from '@components';
import { usePerfilScreen } from './usePerfilScreen';

export const PerfilScreen: React.FC = () => {
  const { usuario, guardar } = usePerfilScreen();

  return (

    {} fullwidth />

  );
};
```

C. Cómo Registrar la Pantalla en la Navegación

Sigue estos 2 sencillos pasos para que TypeScript garantice rutas 100% seguras:

1. Declara la ruta y sus parámetros en `src/core/types/navigation.ts` y `routes.ts`:

```
// src/core/types/navigation.ts
export type RootStackParamList = {
  Home: undefined;
  Details: { id: string; title?: string };
  Perfil: undefined; // <-- Tu nueva ruta
};
```

2. Añade el Screen a `src/navigation/RootNavigator.tsx`:

5. Compilación a APK y Producción (EAS Build)

La plantilla ya incluye `eas.json` configurado para compilar directamente instaladores de Android (APK) e iOS sin necesidad de configurar Gradle o Xcode manualmente:

Objetivo	Comando a Ejecutar	Resultado
Generar APK directo (Android)	<code>npx eas build -p android --profile preview</code>	Enlace de descarga de APK listo para instalar en cualquier teléfono Android.
Compilación Producción (AAB / IPA)	<code>npx eas build -p android --profile production</code>	Paquete firmado listo para subir a Google Play Store o App Store.
Ejecución local en emulador	<code>npx expo run:android</code> o <code>npx expo run:ios</code>	Ejecución nativa en emuladores locales.

6. Resumen de Reglas de Oro SOLID

1. **Nunca importes AsyncStorage directo en una pantalla o componente:** Usa siempre `storageService` que implementa `IStorageService`.
2. **Mantén las pantallas bajo 100-150 líneas:** Toda lógica que no sea renderizar JSX debe ir en un custom hook (`useXScreen.ts`).
3. **Usa los componentes reutilizables de UI:** Evita crear botones o inputs desde cero; usa `AppButton`, `AppTextInput`, `ScreenContainer` y `AppCard`.
4. **Utiliza Path Aliases (@/...):** Escribe `import { AppButton } from '@/components'` en lugar de rutas relativas como `../../components`.

■ REFERENCIA RÁPIDA

Ubicación de la Plantilla: `c:\VibeCoding\AppFacturas\mobile-starter-template`

Script Generador: `node mobile-starter-template/create-app.js "NombreApp" com.paquete.app`

Documentación Digital: `mobile-starter-template/README.md`